Murdoch University

Project 2

TurtleBot3 Wall-Following Report

Unite code: ENG612

Unit name: Autonomous Systems

(S2, 2026)

Date: 26/10/2026

Submitted by,

Kazi Ragib Yeahsar (35502987)

Mir Tahir Tahsin Ahmad (35624701)

Table of Contents

Table of Contents 2
List of Figures 2
List of Tables 2
1 Introduction and objectives 3
2 System architecture 3
3 Laser scan processing 4
4 Control design and tuning strategy 5
5 Simulation and evaluation set-up 6
6 Results 7
7 Simulation-to-real strategy 8
8 Difficulties and how we investigated them 10
9 Limitations 10
10 Github Repository 11
11 Lessons learned and teamwork 11
12 GenAI Acknowledgement 11
13 References 12

List of Figures

Figure 1: System architecture 3
Figure 2: . Laser-measured distance to the wall on straights. Left: Normal PID speed at 0.05 m/s. Right: Advanced PID at 0.065 m/s. The dips are the first moments after each corner. 8
Figure 3: The Gazebo box (left) and the real 2 m × 2 m box with the robot at its start position (right). 9

List of Tables

Table 1: Laser scan table 4
Table 2: Parameters, we have used 6
Table 3: Distance error in Gazebo, target 0.30 m. 7
Table 4: Distance error measured by the laser on the real robot, target 0.30 m. 7
Table 5: What changed between simulation and the real robot. 9

1 Introduction and objectives

Our wall-follower maintains a distance of 0.30 meters from the wall on the left side of our TurtleBot3 Burger within an accuracy of a few millimeters on straight paths in Gazebo and in reality, whereas the maximum deviation is up to 7.4 centimeters.

Project 2 asks for a ROS 2 Humble node that reads the laser scan, estimates the distance to a wall and holds 0.30 m with PID control, first in a Gazebo copy of the laboratory terrain and then on the real robot [1] [2]. The robot follows the left wall of a 2 m × 2 m box, so besides the straight sections it must handle four corners.

A single ROS 2 package called wall_follower was created and three types of control strategies were compared, which have common code:

- **Normal PID** (mode:=baseline): a single PID controller for the steering and fixed forward speed.
- **Rule-based speed** (mode:=extended): the same PID controller for the steering with the forward speed calculated based on rules from front distance and wall-distance error.
- **Advanced PID** (mode:=extended_pid): the same steering PID plus a second PID that controls forward speed. This is our final design.

Distances are measured using the robot's laser unless otherwise specified, and do not include cornering in place, in which the robot does not follow a wall.

2 System architecture

One controller node turns laser scans into velocity commands; a second node only listens and records.

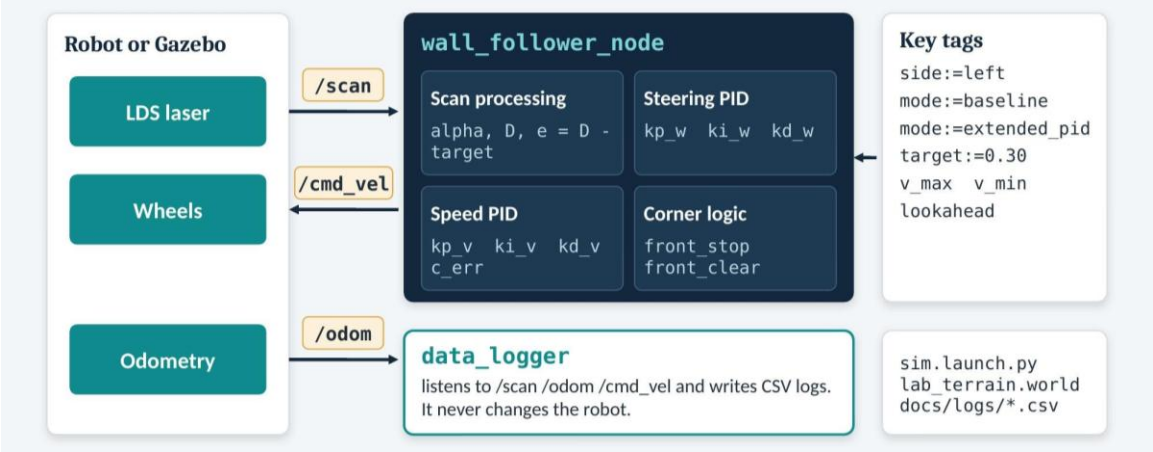


Figure 1: System architecture

Inside wall_follower_node, scan processing feeds the steering PID, the speed PID and the corner logic, and the result leaves as a Twist message on /cmd_vel. The logger data_logger subscribes to /scan, /odom and /cmd_vel and writes CSV files, so measuring cannot change how the robot behaves. The same node runs unchanged in Gazebo [3] and on the real robot; only the source of /scan and /odom changes. In simulation, sim.launch.py starts Gazebo with our world

lab_terrain.world. On the real robot, robot.launch.py from turtlebot3_bringup runs on the Raspberry Pi, while the controller and logger run on a laptop.

The package contains:

- wall_follower_node.py: scan processing, the control designs and the corner logic.
- data_logger.py: records position, commanded velocity and the distance error against the known box geometry.
- launch/sim.launch.py and worlds/lab_terrain.world: the Gazebo set-up.
- scripts/analyze_runs.py: computes the metrics of Section 6 from the logs.

3 Laser scan processing

The node gets both the distance to the wall and the robot's angle to it from two median-filtered side beams, which is why it behaves the same with the simulated and the real laser.

Bearings follow the ROS convention: 0° is straight ahead and angles increase counter-clockwise, so the left wall is at 90°. A beam is found from its bearing using angle_min and angle_increment of each message, not a fixed index, because the real LDS-02 returns about 234 beams per scan and its angle_min changes from scan to scan.

Beam	Bearing	Window	Used for
Perpendicular, b	90°	7 beams	distance to the wall
Ahead, a	45°	7 beams	angle to the wall
Front	0°, 15°, 345°	17, 9 and 9 beams; minimum of the three medians	corner detection, speed

Table 1: Laser scan table

Each beam value is the median of its window. Readings that are not finite or are below 0.05 m, which the real laser produces as dropouts, are replaced by the maximum range before the median is taken, so one bad reading cannot pull the result toward zero. With L as a look-ahead distance (0.2 m in our runs), the wall angle α, the projected distance D and the error e are:

$$\alpha = \operatorname{atan2}\left(a\cos 45^\circ - b,\ a\sin 45^\circ\right)$$
$$D = b\cos\alpha + L\sin\alpha$$
$$e = D - 0.30$$

This term anticipates where the robot would be after L, meaning that the control system anticipates the drift in heading even before the measured distance changes. The robot takes the wall as lost when both a and b are more than 1.0 meters; this means that the robot moves forward slowly and approaches the wall.

The time step for the derivative term comes from the scan's header timestamp, not from the computer's clock. On the real robot, Wi-Fi delivers scans in bursts at about 9.7 Hz on average, and

a clock-based step made the derivative spike. Scans with a timestamp not newer than the previous one are ignored, and the step is capped at 0.5 s.

4 Control design and tuning strategy

Steering is always a PID on the wall-distance error e; the three designs differ only in how forward speed is chosen [4].

The steering PID outputs an angular velocity ω, limited to ±1.5 rad/s, with the integral clamped to ±0.5 to prevent windup. A sign s (+1 for the left wall) makes a distance that is too large always turn the robot toward the wall. Here d_f is the front distance, d_{stop} the corner trigger distance, K_p, K_i, K_d the steering gains, and K_{pv}, K_{iv}, K_{dv} the speed gains. The gains were tuned by trial in Gazebo, one value per run. $K_p = 3.0$ was raised until the robot corrected quickly, $K_d = 0.8$ was added to damp the side-to-side swing, and $K_i = 0$ because the straights were already within millimetres.

$$\omega = s\left(K_p e + K_i \int e\,dt + K_d \frac{de}{dt}\right)$$
$$e_v = (d_f - d_{stop}) - c\,|e|$$
$$v = \text{clamp}\,(\text{PID}_v(e_v),\ v_{\min},\ v_{\max})$$

- **Normal PID:** constant speed; this is the simplest variant.
- **Rule-based speed:** speed decreases if the front distance decreases or if the distance error increases. This is a simple proportional control with a maximum, not a PID control of second level.
- **Advanced PID:** a PID control of second level where the error e_v is the front clearance minus c times the wall distance error, hence the robot decreases speed if the front distance decreases or the robot deviates from the target point. The derivative of this PID is low-pass filtered (with factor 0.3), and it is reset during in-place turns.

Corner handling. When the front distance falls below d_{stop}, the robot stops and rotates in place away from the wall at 0.8 rad/s. It keeps rotating until the front distance exceeds 1.2 m. These two thresholds form a hysteresis; without it the robot stopped turning as soon as the front cleared d_{stop} and drove back toward the wall (Section 8).

Settings. Values not shown in our run commands are the defaults in the code.

Parameter	Gazebo	Real robot
$v_{\max}$ (m/s)	0.08	0.065 (0.05 also tested)
$v_{\min}$ (m/s)	0.04	0.02
Steering K_p, K_i, K_d	3.0, 0, 0.8	3.0, 0, 0.8
Speed K_{pv}, K_{iv}, K_{dv}	0.5, 0, 0.15	0.5, 0, 0.15
Error weight c	2.0	2.0
Look-ahead L (m)	0.2	0.2
d_{stop} (m)	0.34	0.34

Table 2: Parameters, we have used

Tuning strategy.

1. Tune in Gazebo first, where runs are fast and the true position is known.
2. Raise K_p until the robot corrects quickly, then add K_d to damp the oscillation. Change one value per run, log the run to a CSV, and score every run with the same script.
3. Fix the corner behaviour before the speed, because the corners produced the largest errors.
4. Add the speed rule, then replace it with the second PID.
5. On the real robot, start slow and raise the speed only while the error stays similar.

Both PIDs have an integral component with anti-windup protection, although our K_i values are 0, so both act as PD controllers. Since the error when driving straight was a few millimeters to begin with, and an integral component is likely to cause overshoot around corners, we didn't bother testing any other values.

5 Simulation and evaluation set-up

Frame of Gazebo, logger, and lab is one where the origin is located in the inside corner of the box with x direction being the wall facing robot at the beginning while y is left wall with yaw zero in +x direction.

The world lab_terrain.world has four static walls whose inner faces lie at x and y equal to 0 m and 2 m. sim.launch.py spawns the TurtleBot3 Burger at (0.5, 1.7) heading along +x, which puts it 0.3 m from the left wall (y = 2). The controller itself uses no map and no odometry. It works only from laser distances, so it is independent of the frame and transfers from Gazebo to the real robot without change. The frame matters only for evaluation, where the logger compares the robot's position with the known box walls.

Two error metrics are employed here. The true position is used for distance estimation in Gazebo. With the real robot, laser data was taken into account due to odometry drift. For both scenarios, the metrics are as follows:

- **RMS** and **95th percentile** of the distance error on straights, while the robot is driving and not turning in place.

- **Worst error** outside the in-place turns.
- **Jitter**: standard deviation of the steering command, as a measure of how much the controller works.
- **Lap time**: four times the mean interval between corner starts.

6 Results

The Advanced PID controls the distance from the wall at a few millimetres of RMS value in straight lines, but the errors in the real robot occur right after the corners.

Gazebo. Table 1 compares the Normal PID, run with our first gains, with the final Advanced PID, using the true position.

Gazebo, left wall, true position	Normal PID (first gains)	Advanced PID
RMS error on straights (mm)	31.1	5.8
95th percentile (mm)	50.3	14.4
Worst error (cm)	5.6	1.8
Forward speed (m/s)	0.080 fixed	0.067 mean
Lap time (s)	79	92

Table 3: Distance error in Gazebo, target 0.30 m.

This gap reflects the whole sequence of changes, not the second PID alone: after the gain and corner-logic changes of Sections 4 and 8, a later Normal PID run, measured with the laser, reached 5.2 mm RMS and a worst error of 3.4 cm with a 79 s lap. The Advanced PID is slower (92 s against 79 s) and tighter.

Real robot. Table 2 gives the laser-measured results of the runs.

Run (max speed, m/s)	RMS (mm)	95th pct (mm)	Worst (cm)	Jitter	Lap (s)
Normal PID (0.05)	7.4	12.4	5.8	0.053	124
Advanced PID (0.05)	3.8	6.5	5.4	0.037	132
Advanced PID (0.065)	4.1	8.2	5.4	0.036	116

Table 4: Distance error measured by the laser on the real robot, target 0.30 m.

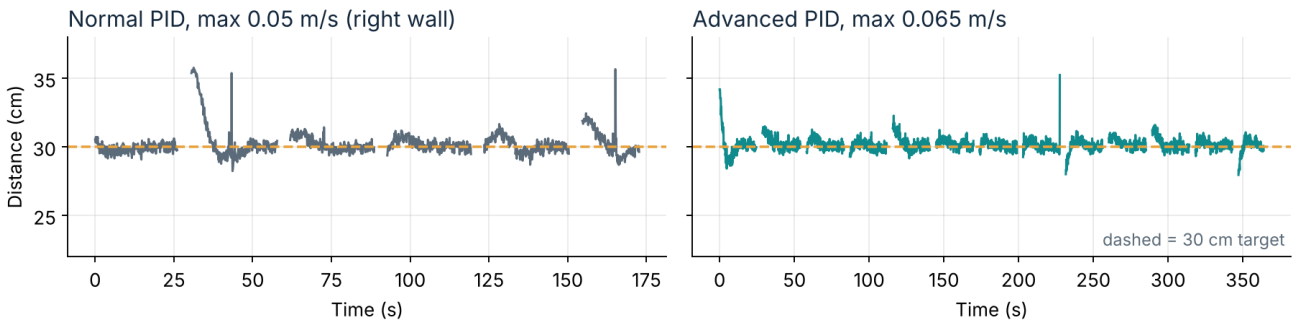


Figure 2: . Laser-measured distance to the wall on straights. Left: Normal PID speed at 0.05 m/s. Right: Advanced PID at 0.065 m/s. The dips are the first moments after each corner.

What the results show:

- **Straights are held within millimetres.** Three of four real runs have 3.8 to 4.1 mm RMS error.
- **Corners are the weak point.** Every dip in Figure 2 is the first second or two after a corner, when the robot is a few centimetres too close to the wall before it settles back to 0.30 m.
- **Speed PID against rule-based speed.** At the same 0.05 m/s limit the Advanced PID gave 3.8 mm against 4.1 mm and a lap of 132 s against 158 s, which is 16% shorter. These are single runs, and the repeat run shows that run-to-run variation is larger than the 0.3 mm difference, so we do not claim the Advanced PID is more accurate.
- **Speed.** Raising the limit from 0.05 to 0.065 m/s shortened the lap from 132 s to 116 s, which is 12%, with the same worst error of 5.4 cm.
- **A repeat run was less accurate.** The repeat at 0.065 m/s had 9.9 mm RMS and a worst error of 7.4 cm, caused by one episode of about 1.2 s around 75 s into the run in which the robot was 5 to 7 cm too close to the wall. We did not identify the cause.

7 Simulation-to-real strategy

Because the controller works only from laser-relative distances, moving from Gazebo to the real robot was a matter of data quality, timing and networking, not of changing the algorithm.

We kept one code base and the same topics, `/scan` and `/cmd_vel`, in both worlds, and changed only parameters on the command line. Table 3 lists what differed and how we handled it.

Aspect	Gazebo	Real robot	What we did
Laser scan	regular scans on simulation time	LDS-02, about 234 beams, angle_min changes per scan, dropouts as non-finite values	find beams by bearing; median over a window; replace invalid readings with the maximum range
Timing	even scan spacing	Wi-Fi delivers scans in bursts, about 9.7 Hz on average	time step from the scan header stamp; ignore out-of-order scans; cap at 0.5 s
Discovery	one machine, default DDS, domain 0	robot and laptop on different subnets, multicast discovery failed	Cyclone DDS [5] unicast peers (robot address and localhost, multicast off), domain 76
Speed	0.08 m/s	0.05 to 0.065 m/s, minimum 0.02 m/s	start slow, then raise the speed while the error stayed similar
Odometry	starts at the spawn pose, accurate	starts where the bring-up began, drifts	logger frame set from the first scans; real-robot error judged from the laser
Stopping	stops at once on Ctrl+C	kept driving after Ctrl+C	disable the default signal handler, publish zero velocity five times on exit

Table 5: What changed between simulation and the real robot.

A placement check makes real runs repeatable. The logger refuses to start until the robot is within 0.10 m of the expected 0.30 m, within 15° of parallel to the wall, and still, and it prints which condition fails; we start the controller only after it confirms the placement.

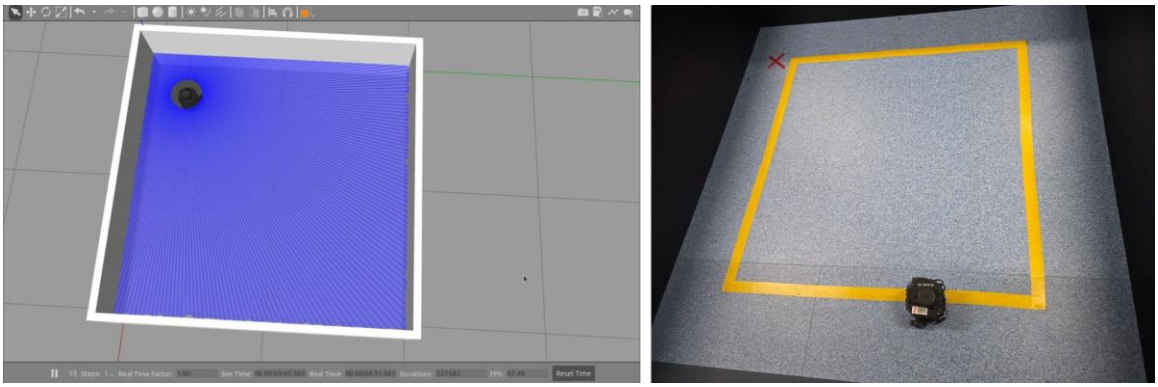


Figure 3: The Gazebo box (left) and the real 2 m × 2 m box with the robot at its start position (right).

The real-robot comparisons are not perfectly controlled. The lab has three boxes of slightly different sizes and we used whichever was free for each test; the demonstration box is 2 m × 2 m.

Because the controller measures distance to the wall directly, box size does not change its control, but it does mean timing figures such as lap time are not strictly comparable across runs.

8 Difficulties and how we investigated them

Six problems shaped the final design, and each was settled by logging the run, finding where in time the error appeared, changing one thing and rerunning.

1. **The robot dived at walls near corners.** It approached walls much closer than 0.30 m, turned, corrected and repeated at the next wall. Our diagnosis from the logs was that the corner rule used a single threshold, so the rotation stopped as soon as the front distance cleared it and the robot drove straight back toward the wall. We added a turning state with two thresholds (start below d_stop, end above 1.2 m), reduced the speed and added the speed control of Section 4. Later runs no longer showed the dive.
2. **The robot kept driving after Ctrl+C.** In ROS 2 Humble, rclpy handles the interrupt itself and shuts the context down before our cleanup code can publish a stop. We initialised rclpy with its signal handler disabled and publish zero velocity five times in the cleanup block. The robot then stopped on every Ctrl+C.
3. **Negative distances in the Gazebo position log.** Our first explanation was wrong. The cause was that Gazebo's odometry already starts at the spawn pose, so adding the start offset again counted it twice. Setting the offsets to zero produced physical values. Because the first diagnosis was wrong, we now check a suspicious column against the known geometry before editing code.
4. **Wobble and spikes on the real robot.** The first real run wobbled and the log had spikes. The scan timestamps arrived unevenly, and our own hands in the laser's view added more spikes. Using the header stamp for the time step, the median filter and keeping clear of the robot removed both.
5. **The laptop could not detect the topic list of the robot.** Since the two computers are in different subnets, the discovery process could not locate anything. The solution to this was using a Cyclone DDS unicast peer file containing the robot's IP address and that of the localhost.
6. **A left-wall run showed errors of 1 to 1.5 m.** The position log put the robot outside the box, which showed it had been placed differently from the start pose we typed into the logger. We rebuilt the logger to take the start pose from the first scans and to wait for a valid placement, and we stopped using that run's position data. On the real robot the odometry also drifted over a lap, so we judge the real robot's error from the laser.

9 Limitations

- **Corner exits are the weak point.** The largest errors in every run occur just after a turn, when the robot is still settling onto the wall. Tuning the exit was not completed.
- **Few repeats.** Each configuration was run once or twice, so we report the values but no confidence intervals. The repeat Advanced PID run at 0.065 m/s was less accurate than the first, from one episode of about 1.2 s, and we do not know its cause.
- **No left-wall Normal PID run on the real robot.** The Normal PID is compared with the Advanced PID in Gazebo only, and on the real robot the Advanced PID is compared with the rule-based speed mode.

- **The Gazebo gap has several causes.** The Normal PID results come from our first gains, so the improvement reflects retuning, the corner logic and the second PID together, not the second PID alone.
- **Ki was not explored.** The integral term was left at zero, so steady offsets from a biased wall estimate were not tested.
- **The real-robot error is the laser's own estimate.** It is what the controller regulates, so it cannot reveal a sensor bias, and we had no external ground truth on the real robot.
- **Boxes vary in size in the lab,** thus lap times in actual laps are just estimates.

10 Github Repository

Code repository. All ROS 2 code, run log and analysis script is available in GitHub repository: https://github.com/ragib-yeahsar/ENG612-Project-2 . The authors of the contributors are Kazi Ragib Yeahsar (GitHub username ragib-yeahsar) and Mir Tahir Tahsin Ahmad (GitHub username jackutsa). Both members committed under their own accounts. This is due to the fact that we shared in one computer, meaning that some of the work was done together and then committed by each individually.

11 Lessons learned and teamwork

1. **Log first, then change code.** All fixes in Section 8 were made by observing when in time the error occurred. Making educated guesses previously took more time than adding the logger.
2. **Simulation moves the algorithm, not the plumbing.** The control law carried over unchanged; scan quality, timing, networking and stopping behaviour did not.
3. **Check the geometry before editing code.** The negative distances and the 1 to 1.5 m errors were frame and placement problems, not controller faults.
4. **Make tests repeatable before comparing them.** The placement checking and constant metrics ensured that future tests would be comparable.
5. **Speed trades against margin.** An increase in speed led to faster laps but less time to compensate at corners, and this explains why speed PID controls the robot by slowing down the robot when front distance is small or the error is large.

We used only one repository: ragib-yeahsar/ENG612-Project-2, in which we made our own commits individually using our individual usernames: Kazi Ragib Yeahsar (ragib-yeahsar), Mir Tahsin (jackutsa). As we used the same computer, some parts of the work were done by both individuals using the same computer and then committed separately.

12 GenAI Acknowledgement

Claude , ChatGPT : To explain and learn the error we are getting from the code, also need for explaining the results we are getting.

13 References

[1] ROBOTIS, "TurtleBot3 e-Manual." [Online], Available: https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/..

[2] Open Robotics, "ROS 2 documentation: Humble Hawksbill." [Online]., Available: https://docs.ros.org/en/humble/.

[3] N. K. a. A. Howard, ""Design and use paradigms for Gazebo, an open-source multi-robot simulator,"," *in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), Sendai, Japan,* p. 2149–2154., 2004.

[4] K. J. Åström and T. Hägglund, "PID Controllers: Theory, Design, and Tuning,. Research Triangle Park," vol. 2nd ed, no. NC, USA: ISA, , 1995.

[5] E. Foundation, " "Eclipse Cyclone DDS."," *[Online]. Available: https://projects.eclipse.org/projects/iot.cyclonedds*..